result in a database. Let's sketch a scenario with a `List[Int]` of items as the source and a fake database implemented as a simple variable, also a `List[Int]`.

There are three stages available to build Akka Streams: `Source`s as producers, `Flow`s as transformations, and `Sink`s as consumers. They are used in both finite and infinite streams; this is a finite example.

10.1.3 Source

In general, a `Source` comes from reading a file, database query, topic, queue, HTTP endpoint, or, as the following example shows, simple collection. The `Source` is lazy: it delays its evaluation until its values are needed. It waits for the consumer to send messages downstream. The combination of the three stages is called a *blueprint* or *graph* and is a pure function. It does nothing until it is asked to `run`, and when it does, it gives back a result. In this chapter, each result has at least a `Future[Done]`. You learn about different results in chapter 16.

The following snippet shows how the `Source` is created with a list of integers and `runForeach` executes the blueprint. This processes each element and stores it in the fake database:

```
class StreamsSpec ... {

  ...

  "a producer of fixed elements 1,2,3 and a function" should {
    "allow when consumed see the side effects | simple test version" in {

      var fakeDB: List[Int] = List()          ❶
      def storeDB(value: Int) =               ❷
        fakeDB = fakeDB :+ value

      val future = Source(List(1, 2, 3))
        .filter(_ % 2 == 0)                   ❸
        .runForeach(i => storeDB(i)))         ❹

      Await.result(future, 1.seconds)
      assert(fakeDB == List(2))
    }
  }

  ...
}
```

❶ Variable representing a side effect

❷ Stores the function

❸ Filters the function

❹ RunForeach creates a consumer for the source and executes the blueprint.

In this simple example, there is already a lot to unpack. You only see a `Source` in this graph, but it has all the stages. Let's look at it again but without the syntactic sugar of the Akka Streams API. You map each stage to a variable individually and then map all of them at once. Let's start with the producer:

```
val producer: Source[Int, NotUsed] = Source(List(1, 2, 3))
```